Section -2

1)**What steps should we take to use PL/SQL records to group related data elements? What is the advantage of using records over individual variables?**

A) In PL/SQL, records are used to group related data elements into a single structure. First, we declare a record type or use %ROWTYPE, then create a record variable, assign or fetch values into it, and access fields using dot notation. Records are better than individual variables because they reduce code complexity, improve readability, and allow us to handle related data as a single unit, especially when working with database rows."

Advantages:

- Groups related data into one variable.
- Reduces the number of variable declarations.
- Makes code more readable and maintainable.
- Easy to fetch and manipulate an entire database row using %ROWTYPE.
- Reduces coding errors and simplifies maintenance

2) **What performance factors should be considered when using the MERGE statement in PL/SQL?**

A) When using the MERGE statement, I ensure that the join columns are properly indexed, use an efficient ON condition, keep table statistics updated, and review the execution plan. For large data loads, I process data in batches and consider the impact of triggers, constraints, and transaction size. These practices help improve the performance of MERGE operations.

3) **I used stored procedure to structure OKL script to perform a complex validation of business rules before inserting data into multiple related tables?**

A) "I used a stored procedure to validate business rules before inserting data into multiple related tables. It performs all required checks, inserts the data only if the validation is successful, and rolls back the transaction if any error occurs, ensuring data consistency and integrity.

4) **Encrypt sensitive data before sending it to external systems through APIs, ensuring confidentiality, integrity, and protection from security threats?**

I secure API communication by using HTTPS/TLS encryption, implementing authentication and authorization, validating requests, and protecting sensitive data. This ensures data confidentiality, integrity, and protection against unauthorized access or tampering while integrating with external systems.

5)

How PL/SQL handle string manipulation and what are some of the key functions used for this purpose?

A) PL/SQL provides many built-in string functions to manipulate text data. These functions are used for searching, extracting, replacing, formatting, and combining strings.

Common string functions:

* LENGTH() – Returns the length of a string.

* SUBSTR() – Extracts part of a string.

* INSTR() – Finds the position of a substring.

* REPLACE() – Replaces one substring with another.

* UPPER() – Converts text to uppercase.

* LOWER() – Converts text to lowercase.

* INITCAP() – Converts the first letter of each word to uppercase.

* TRIM() – Removes leading and trailing spaces.

* LTRIM() / RTRIM() – Removes spaces or specified characters from the left or right.

* CONCAT() or || – Joins two or more strings.

* LPAD() / RPAD() – Pads a string to a specified length.

3-section:

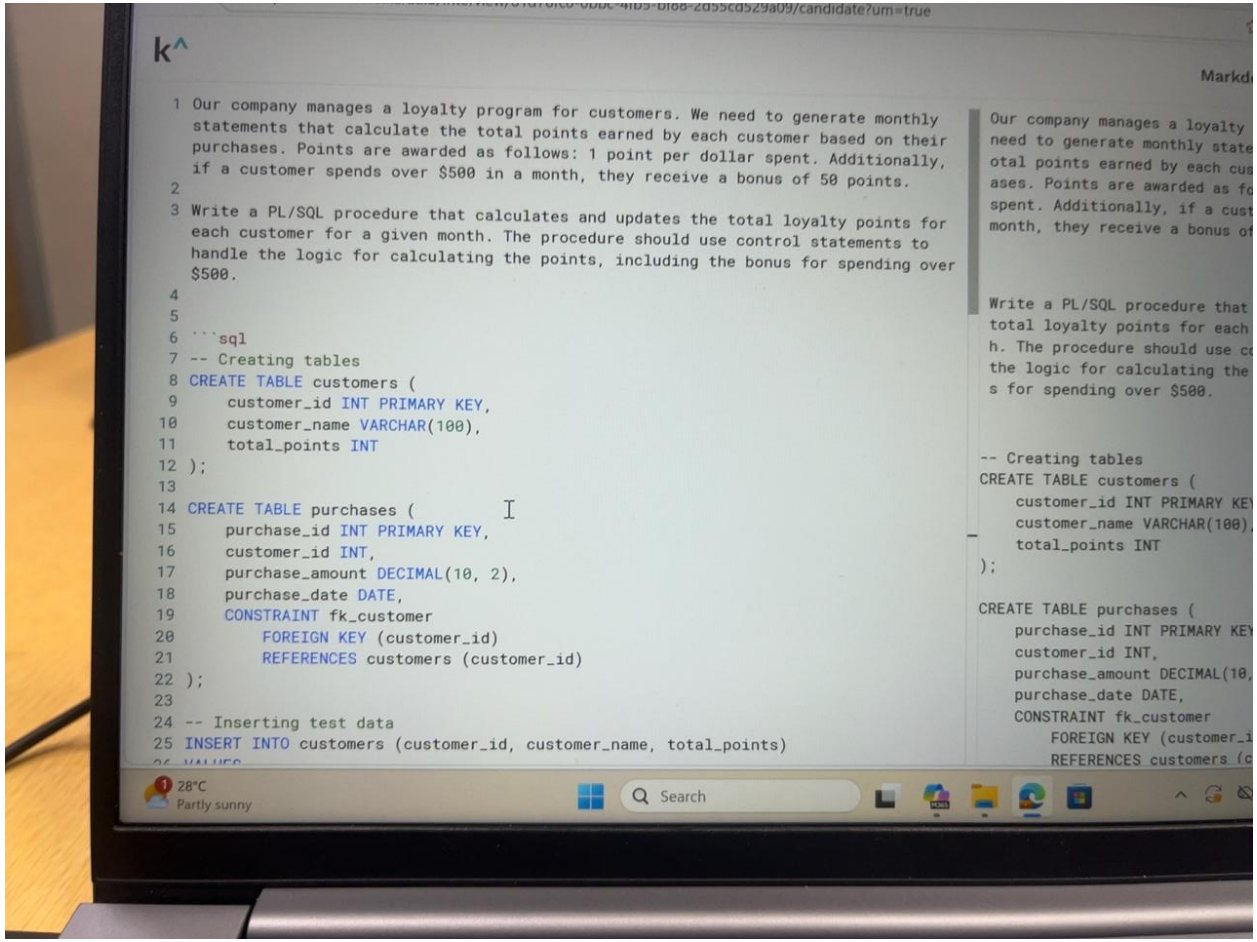
Implement a PL/SQL trigger to update the account balance whenever a new transaction (debit or credit) is inserted. If the balance goes below zero, raise an exception?

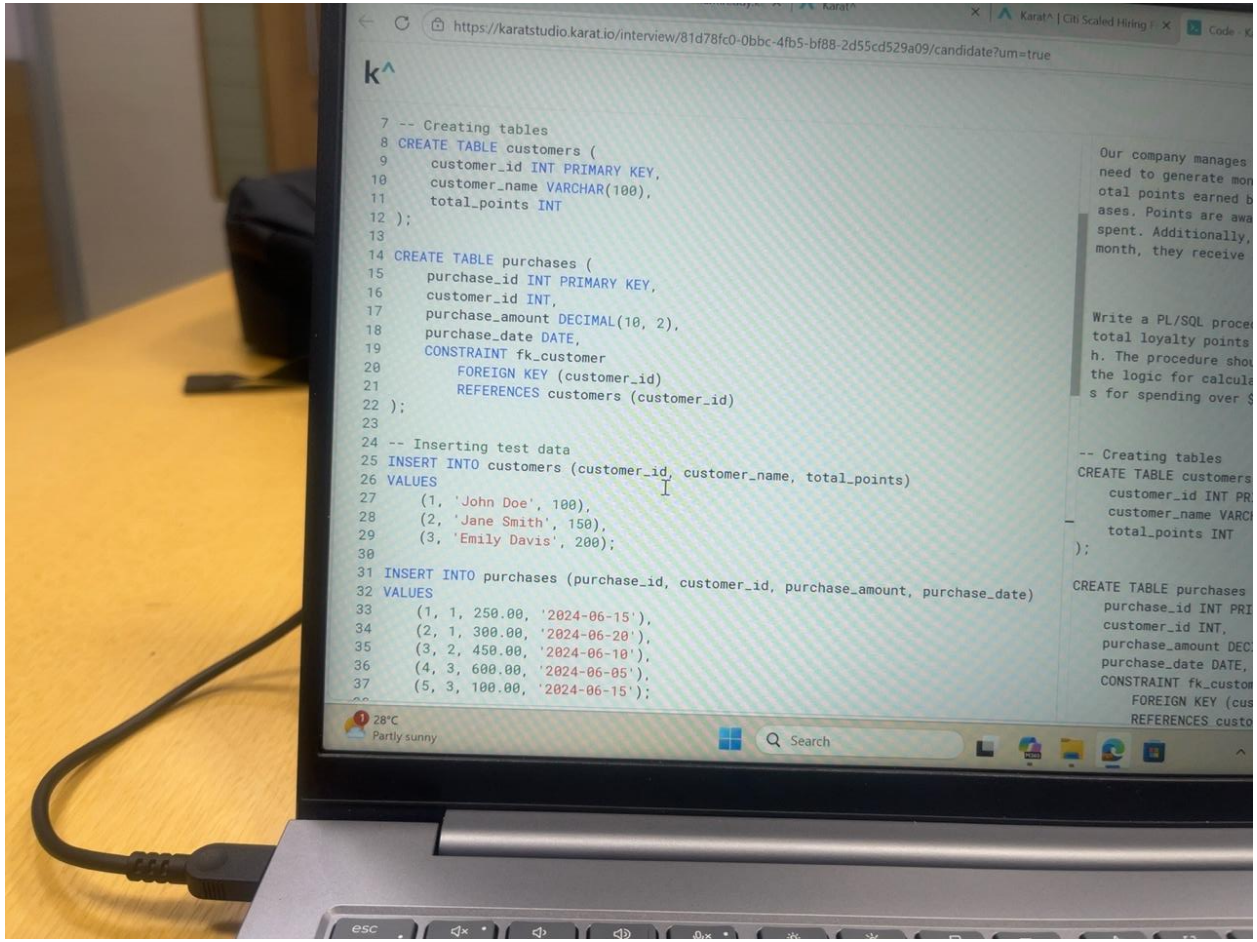
```
CREATE OR REPLACE TRIGGER trg_update_balance
AFTER INSERT ON transactions
FOR EACH ROW
DECLARE
  v_balance accounts.account_balance%TYPE;
BEGIN
  -- Get current balance
  SELECT account_balance
  INTO v_balance
  FROM accounts
  WHERE account_id = :NEW.account_id
  FOR UPDATE;

  IF :NEW.transaction_amount > 0 THEN
    -- Credit transaction
    UPDATE accounts
    SET account_balance = account_balance + :NEW.transaction_amount
    WHERE account_id = :NEW.account_id;

  ELSE
    -- Debit transaction
    IF v_balance + :NEW.transaction_amount < 0 THEN
      RAISE_APPLICATION_ERROR(
```

```
            -20001,
            'Insufficient balance. Transaction rejected.'
        );
    ELSE
        UPDATE accounts
        SET account_balance = account_balance + :NEW.transaction_amount
        WHERE account_id = :NEW.account_id;
    END IF;
  END IF;
END;
/
```


k^
Markd
1 Our company manages a loyalty program for customers. We need to generate monthly statements that calculate the total points earned by each customer based on their purchases. Points are awarded as follows: 1 point per dollar spent. Additionally, if a customer spends over $500 in a month, they receive a bonus of 50 points.
2
3 Write a PL/SQL procedure that calculates and updates the total loyalty points for each customer for a given month. The procedure should use control statements to handle the logic for calculating the points, including the bonus for spending over $500.
4
5
6 ```sql
7 -- Creating tables
8 CREATE TABLE customers (
9 customer_id INT PRIMARY KEY,
10 customer_name VARCHAR(100),
11 total_points INT
12);
13
14 CREATE TABLE purchases (
15 purchase_id INT PRIMARY KEY,
16 customer_id INT,
17 purchase_amount DECIMAL(10, 2),
18 purchase_date DATE,
19 CONSTRAINT fk_customer
20 FOREIGN KEY (customer_id)
21 REFERENCES customers (customer_id)
22);
23
24 -- Inserting test data
25 INSERT INTO customers (customer_id, customer_name, total_points)
28°C
Partly sunny
Search


https://karatstudio.karat.io/interview/81d78fc0-0bbc-4fb5-bf88-2d55cd529a09/candidate?um=true
k^
-- Creating tables
CREATE TABLE customers (
customer_id INT PRIMARY KEY,
customer_name VARCHAR(100),
total_points INT
);
CREATE TABLE purchases (
purchase_id INT PRIMARY KEY,
customer_id INT,
purchase_amount DECIMAL(10, 2),
purchase_date DATE,
CONSTRAINT fk_customer
FOREIGN KEY (customer_id)
REFERENCES customers (customer_id)
);
-- Inserting test data
INSERT INTO customers (customer_id, customer_name, total_points)
VALUES
(1, 'John Doe', 100),
(2, 'Jane Smith', 150),
(3, 'Emily Davis', 200);
INSERT INTO purchases (purchase_id, customer_id, purchase_amount, purchase_date)
VALUES
(1, 1, 250.00, '2024-06-15'),
(2, 1, 300.00, '2024-06-20'),
(3, 2, 450.00, '2024-06-10'),
(4, 3, 600.00, '2024-06-05'),
(5, 3, 100.00, '2024-06-15');
Our company manages
need to generate mon
otal points earned b
ases. Points are awa
spent. Additionally,
month, they receive
Write a PL/SQL proce
total loyalty points
h. The procedure sho
the logic for calcula
s for spending over $
-- Creating tables
CREATE TABLE customers
customer_id INT PR
customer_name VARC
total_points INT
);
CREATE TABLE purchases
purchase_id INT PRI
customer_id INT,
purchase_amount DEC
purchase_date DATE,
CONSTRAINT fk_custom
FOREIGN KEY (cus
REFERENCES custo
28°C
Partly sunny
Search
esc

A) CREATE OR REPLACE PROCEDURE update_loyalty_points

IS

v_spend NUMBER;

BEGIN

FOR r IN (

SELECT customer_id

```
    FROM customers
  )
  LOOP
    SELECT NVL(SUM(purchase_amount),0)
    INTO v_spend
    FROM purchases
    WHERE customer_id = r.customer_id;

    IF v_spend > 500 THEN
      UPDATE customers
      SET total_points = v_spend + 50
      WHERE customer_id = r.customer_id;
    ELSE
      UPDATE customers
      SET total_points = v_spend
      WHERE customer_id = r.customer_id;
    END IF;
  END LOOP;

  COMMIT;
END;
/
```